

Prueba Técnica: Modernización de Arquitectura Core Bancario

Rol: Analista de Sistemas

Duración Sugerida: 4 - 5 horas

Formato de Entrega: Documento PDF o Presentación Ejecutiva + Diagramas

1. Objetivo

Evaluar la madurez técnica y funcional del candidato en el dominio bancario. Se busca verificar su capacidad para diseñar sistemas de alta disponibilidad, garantizar la integridad financiera (ACID), manejar la contabilidad automática y modernizar sistemas legados.

2. Escenario: "Proyecto NeoBank Transformation"

Contexto: El banco tradicional "FinBank" está lanzando una marca digital (Neobanco) dirigida a un segmento joven. Actualmente, "FinBank" opera con un Core Bancario monolítico (legacy) basado en Mainframe que procesa los cierres diarios (EOD - End of Day) en 6 horas.

El negocio requiere lanzar un nuevo **Módulo de Cuentas de Ahorro de Alto Rendimiento** con las siguientes características críticas que el sistema actual no soporta eficientemente:

Requerimientos de Negocio (High-Level):

1. **Apertura de cuenta 100% digital (Onboarding):** Debe incluir validación de identidad y listas negras (AML/KYC) en tiempo real.
2. **Cálculo de Intereses en Tiempo Real:** El cliente debe poder ver cuánto interés ha ganado su dinero al momento de la consulta (devengamiento diario, capitalización mensual).
3. **Transaccionalidad 24/7:** El sistema no puede detenerse durante el cierre diario (EOD).
4. **Integración con Pagos Inmediatos:** Capacidad de recibir y enviar transferencias interbancarias en menos de 10 segundos.
5. **Motor Contable:** Cada transacción monetaria debe generar su partida contable de doble entrada (Debe/Haber) en el Libro Mayor (General Ledger) de forma síncrona o asíncrona segura.

3. Tareas a Desarrollar

El candidato debe abordar los siguientes puntos en su entrega:

Parte 1: Análisis y Descomposición Funcional

- **Matriz de Requerimientos:** Traduce los requerimientos de negocio a requerimientos técnicos. Diferencia claramente entre el flujo transaccional (el movimiento de dinero) y el flujo contable (el registro en el libro mayor).
- **Diagrama de Casos de Uso Críticos:** Enfócate en el flujo: *"Recepción de una transferencia interbancaria entrante con validación de cuenta y abono de saldo"*.

Parte 2: Diseño de Arquitectura y Datos (Core)

- **Arquitectura del Sistema:** Propón un diagrama de arquitectura (Microservicios, Modular Monolith o SOA).
- **Modelo de Datos (ERD Preliminar):** Diseña las tablas esenciales para manejar las cuentas y los saldos.
- **Estrategia de Coexistencia:** ¿Cómo convive este nuevo módulo con el Core Legacy del banco principal para la consolidación financiera?

Parte 3: Tecnologías y Patrones

- **Stack Tecnológico:** Define Lenguaje, Base de Datos y Middleware.
- **Manejo de Transacciones Distribuidas:** Si eliges microservicios, explica cómo manejarás una falla si el dinero se debita pero el servicio de contabilidad falla.

Parte 4: Procesos Batch y EOD (End of Day)

- **Optimización del Cierre:** El negocio exige operación 24/7. Describe cómo diseñarías el proceso de "Cierre Diario" (capitalización de intereses, cobro de comisiones, generación de extractos) sin detener la base de datos ni impedir que los clientes usen sus tarjetas durante ese proceso.

Parte 5: Seguridad y Cumplimiento

- **Seguridad Transaccional:** Explica cómo implementarás **Idempotencia** en las APIs de transferencia para evitar que un reintento de red duplique un depósito.
- **Auditoría:** Diseña un mecanismo de traza inmutable para que auditoría interna pueda reconstruir la historia de una cuenta ante un reclamo.

Desarrollo de la prueba

PARTE 1: Análisis y Descomposición Funcional

1-1 Matriz de requerimientos

Requerimiento de Negocio	Requerimiento Técnico	Observaciones
Apertura de cuenta	Uso de API de Listas negras para validar al cliente antes de proceder con su apertura de cuenta	Sincronía obligatoria
Cálculo de intereses	Creación de proceso que se encargue de generar el devengamiento de intereses diarios	Debe de ser un proceso independiente del Cierre del día
Operaciones 24/7	Contar con un Sistema que esté operativo durante todo el día	El proceso del cierre del día no debe de generar bloqueos en las tablas que son consultadas por el Sistema
Integración de pagos inmediatos	Uso de API para envío y recepción de pagos inter bancarios	Se debe de validar que la cuenta tenga saldo suficiente para realizar transferencias a cuentas de otros bancos, cada transacción de transferencia no debe tomar más de 10 segundos
Contabilidad Automática	Contabilidad con doble entrada por cada transacción monetaria	Flujo separado del Saldo

1.2 Diagrama de casos de uso críticos

Recepción de una transferencia interbancaria entrante con validación de cuenta y abono de saldo

Actores:

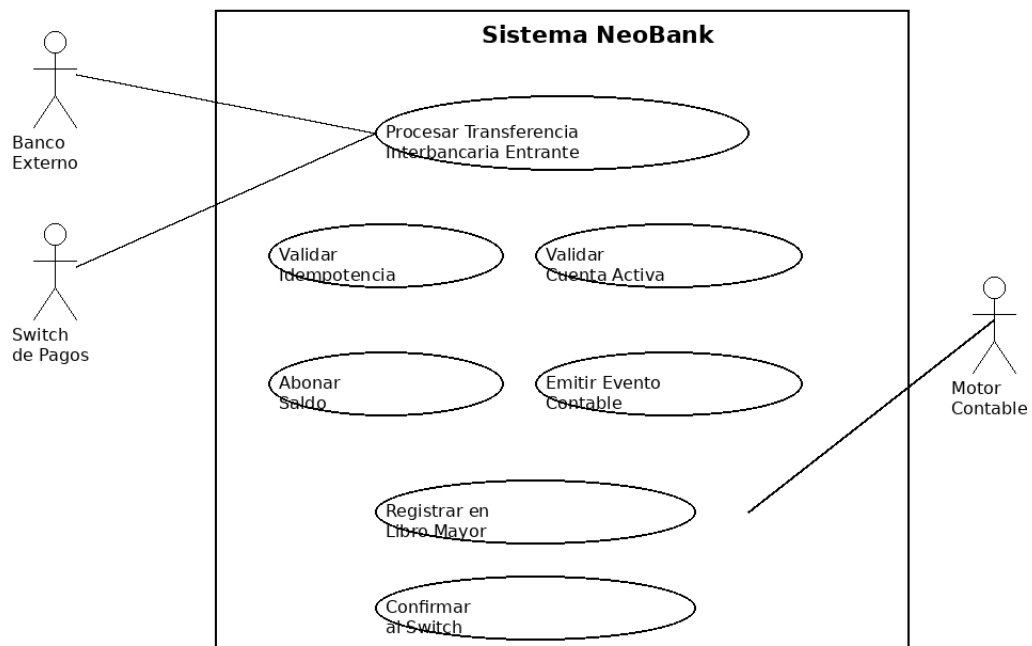
Banco Externo

Switch de Pagos

Sistema NeoBank

Motor Contable

Diagrama de Caso de Uso - Transferencia Interbancaria Entrante



Parte 2: Diseño de Arquitectura y Datos (Core)

2.1 Arquitectura del Sistema

Microservicios con Arquitectura basada en eventos

Servicio de Cuentas

Servicio de Saldos

Servicio de Transacciones

Motor de Cálculo de intereses

Servicio de Contabilidad – Libro mayor

Onboarding Digital / Prevención de Lavado de Activos (AML) y Conozca a su Cliente (KYC)

Módulo de Reportes y Estados de Cuenta

2.2 Modelo de Datos (ERD Preliminar)

Tablas preliminares

Cliente

Cuenta

Producto

Saldo_Cuenta

Transaccion

Asientos

CLIENTE
cliente_id pk
nombre
documento

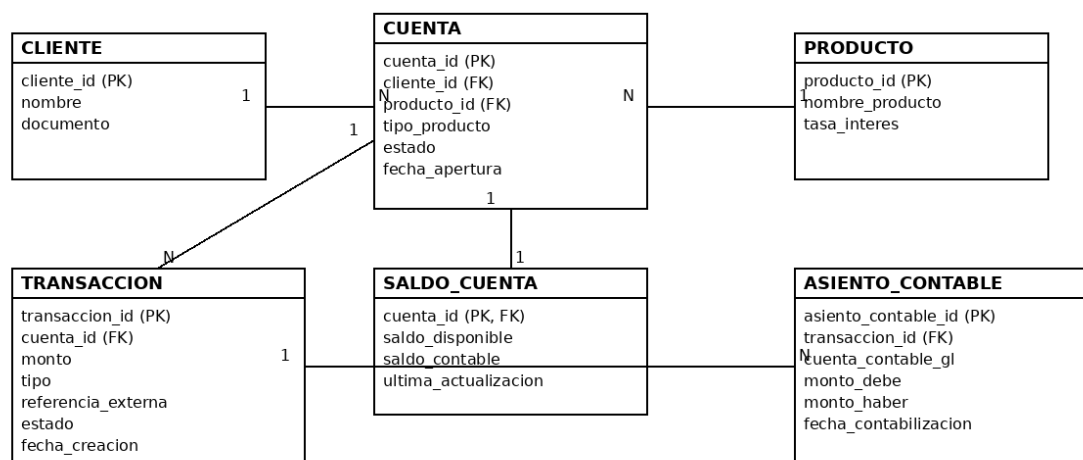
CUENTA
cuenta_id pk
cliente_id fk
producto_id fk
tipo_producto
estado
fecha_apertura

PRODUCTO
producto_id pk
nombre_producto
tasa_interes

SALDO_CUENTA
Cuenta_id pk
saldo_disponible
saldo_contable
ultima_actualizacion

TRANSACCION
transaccion_id pk
cuenta_id fk
monto
tipo
referencia_externa
estado
fecha_creacion

ASIENTO_CONTABLE
asiento_contable_id pk
transaccion_id fk
cuenta_contable_gl
monto_debe
monto_haber
fecha_actualizacion



2.3 Estrategia de Coexistencia

Patrón Strangler Fig Pattern

El NeoBank mantiene su propio libro auxiliar

Se envían asientos consolidados diarios al Core Legacy

El Core sigue siendo el sistema regulatorio oficial

El mainframe no participa en tiempo real, solo en consolidación

Parte 3: Tecnologías y Patrones

3.1 Stack Tecnológico Propuesto

Capa	Tecnología
Back End	Java 17 / Spring Boot
Base de Datos	PostgreSQL (ACID)
Contabilidad	PostgreSQL + particionado
Mensajería	Apache Kafka
API	REST + JSON
Seguridad	OAuth2 / mTLS
Observabilidad	Prometheus + Grafana

3.2 Manejo de transacciones distribuidas

Casuística:

El saldo se acredita pero falla la contabilidad

Posibles causas:

- El servicio de contabilidad está caído
- Hay un timeout
- Error de red
- Error en una validación contable

Solución:

Utilizar una secuencia de transacciones locales, coordinadas por eventos, con acciones compensatorias si algo falla.

Flujo:

Paso 1: Se acredita el saldo con una transacción local

Servicio de Saldo:

- Valida idempotencia
- Actualiza saldo

- Commit exitoso

Paso 2: Se emite un evento de transacción enviada

Este evento contiene datos de:

transaccion_id

cuenta_id

monto

tipo

fecha_hora

Paso 3: Servicio de libro mayor consume el evento

Servicio de libro mayor:

- Consume la transacción enviada
- Genera asientos contables
- Guarda el asiento del libro mayor

Paso 4: ¿Qué pasa si falla la contabilidad?

Se procede con los reintentos automáticos

- Kafka reintenta
- El consumidor vuelve a procesar
- Idempotencia evita duplicados

Si después de n intentos fallas se procede con Dead letter queue que permite que:

- No bloquear el flujo principal
- Alertar a operaciones
- Reprocesar manualmente

El dinero **no se pierde**, solo está pendiente de contabilizar.

Paso 5: Proceder con la consistencia eventual

- El saldo se acredita **inmediatamente**
- La contabilidad puede tardar segundos o minutos
- Al final, **todo cuadra**

Paso 6: En caso de ser necesario proceder con la compensación

Si se trata de un error irrecuperable se procede a realizar:

- Revertir saldo
- Emitir evento compensatorio
- Registrar ajuste contable

Parte 4: Procesos Batch y EOD (End of Day)

Objetivo:

Permitir **operación 24/7**, sin “bajar el sistema” para:

- Capitalizar intereses
- Cobrar comisiones
- Generar estados de cuenta

1- Escenario actual de ejemplo:

23:00 → Se bloquean transacciones



Cálculo de intereses



Cobro de comisiones



Generación de extractos



Cuadres contables



05:00 → Sistema vuelve a operar

Problemas detectados:

- No es 24/7
- Imposible realizar pagos inmediatos
- Riesgo operativo alto

2- Escenario sugerido:

Los procesos de cierre NO afectan la operación online, esto se lograría con:

- Desacoplamiento
- Eventos
- Versionado de datos
- Lecturas consistentes

Procesos desacoplados

Cada tarea del EOD es un proceso independiente:

Capitalización de intereses

- No recalcula todo
- Consume intereses ya devengados

Generación de extractos

- Lee snapshots históricos
- No bloquea saldos actuales

Cobro de comisiones

- Se ejecuta como evento
- Puede ser inmediato o diferido

Si alguno de estos procesos falla el cierre no se cae

3- Arquitectura: Event-driven

Se utilizarán Eventos → Procesos especializados

Ejemplo de eventos:

- Intereses devengados diariamente
- Capitalización mensual de intereses
- Estado de cuenta generado
- Comisión cobrada

Cada servicio consume eventos, ejecuta su lógica, registra resultado

4- Snapshots por fecha

Ejemplo:

Cuenta	Fecha	Ledger balance
123	12/01/2026	50.00

Los procesos EOD **leen snapshots**, no el saldo vivo.

5- MVCC – Lecturas consistentes

MVCC (Multi-Version Concurrency Control)

- Lectores no bloquean escritores
- Escritores no bloquean lectores
- Cada proceso ve su “versión” de los datos

Mientras:

- Cliente transfiere dinero
- El EOD puede leer el saldo de ayer

Ambos ocurren en paralelo

6- Intereses

Devengamiento diario (online)

Cada día:

- Se calcula interés diario
- Se guarda como dato acumulado

NO se abona al saldo todavía

Capitalización mensual (evento)

El último día del mes:

Evento: Capitalización de intereses mensual

→ Suma intereses devengados

→ Abona al saldo

→ Genera asiento contable

Mientras tanto:

- El cliente sigue transfiriendo
- Usando tarjeta
- Consultando saldo

7 - ¿Qué ve el cliente?

Durante el mes:

“Intereses generados hasta hoy: S/ 18.75”

Fin de mes:

“Capitalización de intereses: + S/ 18.75”

Estas operaciones son transparentes y en tiempo real

8 - ¿Qué ve contabilidad?

- Asientos diarios de devengo
- Un asiento mensual de capitalización
- Cuadre perfecto contra ledger

Parte 5: Seguridad y Cumplimiento

5.1 Seguridad transaccional

Objetivo:

Garantizar que **una transferencia se procese una sola vez**, aunque:

- El cliente reintente
- La red falle
- El switch reenvíe el mensaje

A- El problema real que resuelve

Escenario típico:

Cliente envía transferencia

↓

Sistema procesa OK

↓

Respuesta se pierde (timeout)



Cliente / Switch reintent

Sin idempotencia:

- Se vuelve a acreditar saldo
- Se generan dos movimientos
- Error financiero grave

Con idempotencia:

- El sistema reconoce el intento
- Devuelve la misma respuesta
- NO duplica el efecto

Tablas y campos relacionados propuestos para validar idempotencia

Tabla Header

Campo

Idempotency-Key

Ejemplo: Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

Características:

- Lo genera el **cliente o switch**
- Es único por intención de negocio
- Se reutiliza en todos los reintentos

Tabla idempotent_request

Campo	Función
key	Identifica la intención única
request_hash	Garantiza que el payload no cambió
status	Estado del procesamiento
response	Respuesta guardada

“La idempotencia garantiza que una intención de negocio se ejecute una sola vez. Usamos un Idempotency-Key persistido y validado transaccionalmente, devolviendo siempre la misma respuesta ante reintentos.”

Intención de negocio: Representa una acción única que debe producir un único efecto financiero.

B- Flujo paso a paso

Paso 1 – Llega la solicitud

POST /transfers

Idempotency-Key: ABC-123

Paso 2 – Se valida idempotencia

1- Buscar key en idempotent_request

2- Casos posibles:

Caso A: NO existe

- Insertar registro:

status = IN_PROGRESS

- Procesar transferencia

Caso B: Ya existe y está PROCESSED

- NO procesar

- Retornar response guardada

Caso C: Existe pero FAILED

- Reintentar

- Rechazar

- Enviar a revisión

Paso 3: Hash del request (protección adicional)

Evita utilizar la misma key pero cambio el monto

Por ejemplo:

request_hash = SHA-256(payload)

Si:

- key igual

- hash diferente

Rechazo inmediato:

409 Conflict – Idempotency violation

Paso 4: Guardado de la respuesta

Una vez procesado correctamente:

status = PROCESSED

response = {

transaction_id: 789,

status: SUCCESS,

amount: 500

}

5.2 Auditoría

Cuando hay un reclamo, el banco debe poder responder con evidencia técnica, contable y legal.

1- Registro de eventos inmutables

¿Qué se audita?

Cada acción que afectó la cuenta

- Transferencias

- Depósitos

- Retiros

- Intereses

- Comisiones

- Reversiones

2- Ledger contable (Libro Mayor y Sub libro mayor)

¿Qué se valida?

- Que cada evento tenga su asiento contable

- Que el Debe = Haber

- Que los asientos no hayan sido modificados

Auditoría cruza:

Evento ↔ Transacción ↔ Asiento contable

Si algo no cuadra el error está identificado

3- Snapshots históricos de saldo

¿Para qué sirven?

- Ver el saldo a una fecha específica
- No depender del saldo actual

Ejemplo:

Fecha	Saldo
2026-01-10	S/ 500
2026-01-11	S/ 1000

4- Hash encadenado (integridad)

¿Qué garantiza?

- Que ningún evento fue alterado
- Que no hubo manipulación posterior

Auditoría:

- Recalcula hashes
- Verifica continuidad

5- Bitácora técnica (Logs de sistema)

Incluye:

- Request/Response
- Idempotency-Key
- Errores
- Reintentos
- Tiempos

Permite saber si:

- Hubo timeout
- Hubo reintento
- Hubo duplicidad

6- Registro de idempotencia (intenciones)

¿Qué se revisa?

- Si hubo una o varias intenciones
- Si el sistema procesó más de una vez
- Si el error fue técnico o funcional

Muy útil en reclamos de:

- Doble crédito
- Doble débito

7- DLQ (Dead Letter Queue)

Auditoría revisa:

- Eventos que no se procesaron
- Si hubo impacto contable
- Si se regularizó correctamente

Importante para:

- Asientos tardíos
- Ajustes contables

8- Control de accesos y trazabilidad humana

¿Quién hizo qué?

- Usuario interno
- Sistema automático
- Proceso batch

Auditoría valida:

- Roles
- Autorizaciones

9- Reprocesos controlados

Auditoría puede:

- Reproducir eventos
- Simular reconstrucción
- Comparar resultados

Evidencia entregada al cliente

- Línea de tiempo clara
- Movimientos exactos
- Justificación contable
- Respaldo técnico